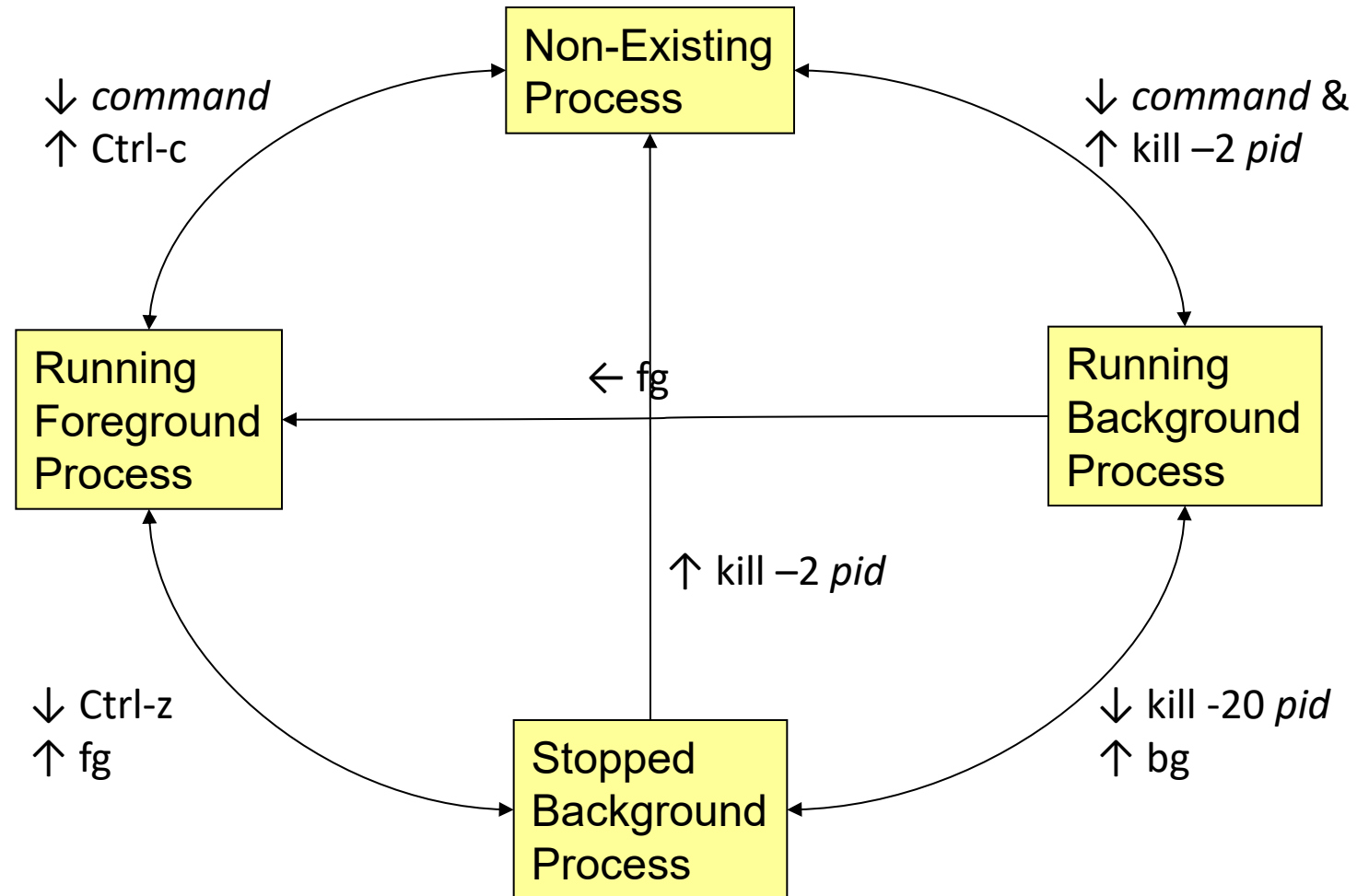


Signals

UNIX Process Control



Process Control Implementation

Exactly what happens when you:

- Type Ctrl-c?
 - Keyboard sends hardware interrupt
 - Hardware interrupt is handled by OS
 - OS sends a 2/SIGINT **signal**
- Type Ctrl-z?
 - Keyboard sends hardware interrupt
 - Hardware interrupt is handled by OS
 - OS sends a 20/SIGTSTP **signal**
- Issue a “kill *—sig pid*” command?
 - OS sends a *sig* **signal** to the process whose id is *pid*
- Issue a “fg” or “bg” command?
 - OS sends a 18/SIGCONT **signal** (and does some other things too!)

Signals

Q1: How does the **OS** communicate to an **application process**?

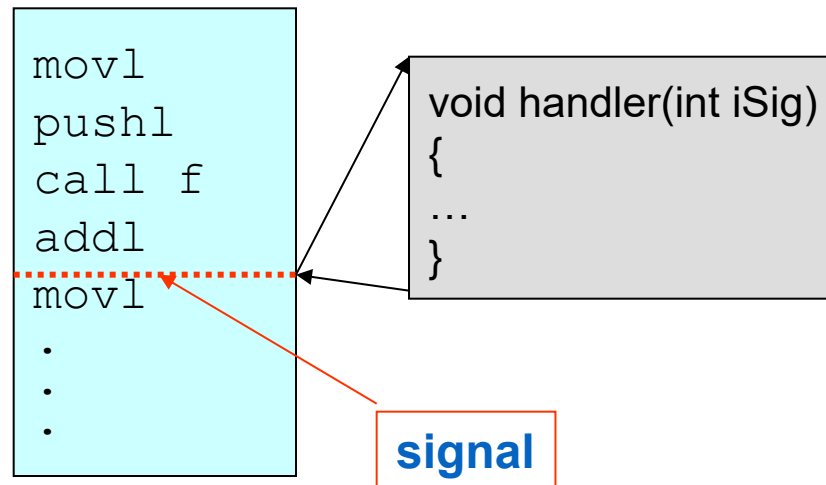
A1: **Signals**

Definition of Signal

Signal: A notification of an event

- Event gains attention of the OS
- OS stops the application process immediately, sending it a signal
- **Signal handler** executes to completion
- Application process resumes where it left off

Process



Examples of Signals

User types Ctrl-c

- Event gains attention of OS
- OS stops the application process immediately, sending it a 2/SIGINT signal
- Signal handler for 2/SIGINT signal executes to completion
 - Default signal handler for 2/SIGINT signal exits process



Process makes illegal memory reference

- Event gains attention of OS
- OS stops application process immediately, sending it a 11/SIGSEGV signal
- Signal handler for 11/SIGSEGV signal executes to completion
 - Default signal handler for 11/SIGSEGV signal prints “segmentation fault” and exits process

Reminder: Kernel Mode

- When the CPU is in *kernel mode*, it is assumed to be executing *trusted* software, and thus it can execute any instructions and reference any memory addresses.
- The *kernel* is the core of the operating system and it has complete control over everything that occurs in the system.
- The kernel is *trusted* software, all other programs are considered *untrusted* software.
- A *system call* is a request to the kernel in a Unix-like operating system by an *active process* for a service performed by the kernel.

Some Definitions

- A **process** is an executing instance of a program. An **active process** is a process that is currently advancing in the CPU (while other processes are waiting in memory for their turns to use the CPU).
- The execution of a process can be interrupted by an **interrupt**.
- An **interrupt** is a **notification to the operating system** that an event has occurred, which results in changes in the sequence of instructions that is executed by the CPU.

Types of Interrupts

- **Hardware interrupts** (also called **external/asynchronous interrupts**), are ones in which the notification originates from a hardware device such as a keyboard, mouse or system clock.
- **Software interrupts** include **exceptions** and **traps**
 - **Exceptions:** triggered by an action of the process without its knowledge (division by zero, access to paged memory, etc.)
 - **Traps:** triggered by the process using system calls
- Usually, no interrupt should be ignored by the OS.

Interrupts:

The basic mechanism

1. The CPU receives the interrupt,
2. Control is transferred to the OS,
3. Current state is saved,
4. The request is serviced,
5. Previous state is restored,
6. Control is returned.

Signals

- Signals are **notifications sent to a process** in order to notify it of various "important" events.
- Signals cause the process to **stop whatever it is doing** at the moment, and force the process to **handle them immediately**.
- The process may configure how it handles a signal.
 - (Except for some signals that it cannot configure)
- **Signals** are different from **interrupts**:
 - Signals are generated by the **OS**, and received and handled by a **process**.
 - Interrupts are generated by the **hardware**, and received and handled by the **OS**.
- Signals and interrupts are both **asynchronous**
- Signals in unix have names and numbers.
 - Use 'man kill' to see the types of signals

Triggers for Signals

- Some examples for signal triggers:
 - Asynchronous input from the user such as ^C (SIGINT), or typing 'kill pid' at the shell
 - The system or another process, for instance if an alarm set by the process has timed out (SIGALRM)
 - An exception in hardware caused by the process, such as Illegal memory access (SIGSEGV)
 - The exception causes a hardware interrupt,
 - The hardware interrupt is received by the OS
 - The signal is generated by the OS and 'sent' to the process
 - A debugger wishing to suspend or resume the process

Sending Signals via Keystrokes

Three signals can be sent from keyboard:

- Ctrl-c → 2/SIGINT signal
 - Default handler exits process
- Ctrl-z → 20/SIGTSTP signal
 - Default handler suspends process
- Ctrl-\ → 3/SIGQUIT signal
 - Default handler exits process

Sending Signals Using the Keyboard

The most common way of sending signals to processes is using the keyboard:

- **Ctrl-C**: Causes the system to send an `INT` signal (`SIGINT`) to the running process.
- **Ctrl-Z**: causes the system to send a `TSTP` signal (`SIGTSTP`) to the running process.
- **Ctrl-**: causes the system to send a `ABRT` signal (`SIGABRT`) to the running process.

Sending Signals from the Command Line

- The `kill` command sends a signal to a process.

```
kill [options] pid
```

`-l` lists all the signals you can send

`-signal` defines the signal to send

- the default is to send a `TERM` signal to the process.

- The `fg` command resumes execution of a process (that was suspended with **Ctrl-Z**), by sending it a `CONT` signal.

Signals

- Allows one process to interrupt another process using OS signaling mechanisms
- A signal is an indication that notifies a process that some event has occurred
- 30 types of signals on Linux systems
- Each signal corresponds to some kind of system event

Signals

- Without signals, low-level hardware exceptions are processed by the kernel's exception handlers only, and are not normally visible to user processes
- Signals expose occurrences of such low-level exceptions to user processes
- If a process attempts to divide by zero, kernel sends a SIGFPE signal (number 8)
- If a process makes an illegal memory reference, the kernel sends it a SIGSEGV signal (number 11)
- If you type a ctrl-C, this sends a SIGINT to foreground process
- A process can terminate another process by sending it a SIGKILL signal (#9)

Signals

Number	Name/Type	Event
2	SIGINT	Interrupt from keyboard
11	SIGSEGV	invalid memory ref (seg fault)
14	SIGALRM	Timer signal from alarm function
29	SIGIO	I/O now possible on descriptor

Signals

- Kernel sends a signal to a destination process by updating some state in the context of the destination process
- A destination process *receives* a signal when it is forced by the kernel to react in some way to the delivery of the signal:
 - can ignore the signal
 - terminate / exit (this is the usual default)
 - *catch* the signal by executing a user-defined function called the *signal handler*

Signals

- A signal that has been sent but not yet received is called a *pending* signal
 - At any point in time, there can be at most one pending signal of a particular type (signal number)
 - A pending signal is received at most once
- a process can selectively block the receipt of certain signals
 - when a signal is blocked, it can be delivered, but the resulting pending signal will not be received until the process unblocks the signal
- For each process, the kernel maintains the set of pending signals in the `pending` bit vector, and the set of blocked signals in the `blocked` bit vector

Signals

- Default action is to terminate a process
- Sending signals with kill function
 - kill -9 PID at command line, or can call kill from within a process
- A process can send SIGALRM signals to itself by calling the alarm function
 - alarm(T seconds) arranges for kernel to send a SIGALRM signal to calling process in T seconds
 - see code example next slide
 - `#include<signal.h>`
 - uses `signal` function to install a signal handler function that is called asynchronously, interrupting the infinite while loop in main, whenever the process receives a SIGALRM signal
 - When handler returns, control passes back to main, which picks up where it was interrupted by the arrival of the signal, namely in its infinite loop

Signals

```
#include <signal.h>
```

```
int beeps=0;
```

```
void handler(int sig) {
```

```
    if (beeps<5) {
```

```
        alarm(3);
```

```
        beeps++;
```

```
    } else {
```

```
        printf("DONE\n");
```

```
        exit(0);
```

```
    }
```

```
}
```

```
int main() {
```

```
    signal(SIGALRM, handler);
```

```
    alarm(3);
```

```
    while(1) { ; }
```

```
    exit(0);
```

```
}
```

Signal handler

cause next SIGALRM to be sent to this process in 3 seconds

register signal handler

cause first SIGALRM to be sent to this process in 3 seconds

infinite loop that gets interrupted by signal handling

Signals

- Receiving signals:
 - when a kernel is returning from some exception handler, it checks to see if there are any pending signals for a process before passing control to the process
 - default action is typically termination
 - `signal(signum, handler)` function is used to change the action associated with a signal
 - if handler is `SIG_IGN`, then signals of type `signum` are ignored
 - if handler is `SIG_DFL`, then revert to default action
 - otherwise handler is user-defined function call the signal handler. This installs or registers the signal handler.
 - Invocation of the signal handler is called *catching the signal*
 - Execution of signal handler is called *handling the signal*

Signals

- When a process catches a signal of type `signum=k`, the handler installed for signal `k` is invoked with a single integer argument set to `k`
- This argument allows the same handler function to catch different types of signals
- When handler executes its return statement (finishes), control usually passes back to the instruction where the process was interrupted

Signals

- Handling multiple signals
 - choose to handle the lower number signals first
 - pending signals are blocked,
 - e.g. if a 2nd SIGINT is received while handling 1st SIGINT, the 2nd SIGINT becomes pending and won't be received until after the handler returns
 - pending signals are not queued
 - there can be at most one pending signal of type k, e.g. if a 3rd SIGINT arrives while the 1st SIGINT is being handled and the 2nd SIGINT is already pending, then the 3rd SIGINT is dropped
 - system calls can be interrupted
 - slow system calls that are interrupted by signal handling may not resume in some systems, and may return immediately with an error

Signals

- Portable signal handling: `sigaction()` is a standard POSIX API for signal handling
 - allows users on Posix-compliant systems such as Linux and Solaris to specify the signal-handling semantics they want, e.g. whether sys call is aborted or restarted
- `sigaction(signum, struct sigaction*act, struct sigaction *oldact)`
- each struct sigaction can define a handler, e.g. `action.sa_handler = handler;`
- In part iv of PA #1,
 - use `sigaction()` to define the handler, e.g. *reschedule()*
 - Also need to set up the timer - use *setitimer* instead of *alarm*. A SIGALRM is delivered when timer expires.

Sending Signals via Commands

kill Command

kill -*signal* *pid*

- Send a signal of type **signal** to the process with id **pid**
- Can specify either signal type name (-SIGINT) or number (-2)
- No signal type name or number specified => sends 15/SIGTERM signal
 - Default 15/SIGTERM handler exits process
- Editorial comment: Better command name would be **sendsig**

Examples

kill -2 1234

kill -SIGINT 1234

- Same as pressing Ctrl-c if process 1234 is running in foreground

Sending Signals via Function Call

`raise()`

```
int raise(int iSig);
```

- Commands OS to send a signal of type `iSig` to current process
- Returns 0 to indicate success, non-0 to indicate failure

Example

```
int ret = raise(SIGINT); /* Process commits suicide. */  
assert(ret != 0);        /* Shouldn't get here. */
```

Note: C90 function

Sending Signals via Function Call

kill()

```
int kill(pid_t iPid, int iSig);
```

- Sends a **iSig** signal to the process whose id is **iPid**
- Equivalent to **raise(iSig)** when **iPid** is the id of current process
- Editorial comment: Better function name would be **sendsig()**

Example

```
pid_t iPid = getpid(); /* Process gets its id.*/  
kill(iPid, SIGINT);    /* Process sends itself a  
                        SIGINT signal (commits  
                        suicide?) */
```

Note: POSIX (not C90) function

Signal Handling

Each signal type has a default handler

- Most default handlers exit the process

A program can install its own handler for signals of any type

Exceptions: A program *cannot* install its own handler for signals of type:

- 9/SIGKILL
 - Default handler exits the process
 - Catchable termination signal is 15/SIGTERM
- 19/SIGSTOP
 - Default handler suspends the process
 - Can resume the process with signal 18/SIGCONT
 - Catchable suspension signal is 20/SIGTSTP

Installing a Signal Handler

signal()

```
sighandler_t signal(int iSig,  
                    sighandler_t pfHandler);
```

- Installs function **pfHandler** as the handler for signals of type **iSig**
- **pfHandler** is a function pointer:

```
typedef void (*sighandler_t)(int);
```
- Returns the old handler on success, **SIG_ERR** on error
- After call, **pfHandler** is invoked whenever process receives a signal of type **iSig**

Installing a Handler Example 1

Program testsignal.c:

```
#define _GNU_SOURCE /* Use modern handling style */
#include <stdio.h>
#include <assert.h>
#include <signal.h>

static void myHandler(int iSig) {
    printf("In myHandler with argument %d\n", iSig);
}
...
```


Installing a Handler Example 1 (cont.)

Program testsignal.c (cont.):

```
...
int main(void) {
    void (*pfRet) (int);
    pfRet = signal(SIGINT, myHandler);
    assert(pfRet != SIG_ERR);

    printf("Entering an infinite loop\n");
    for (;;)
        ;
    return 0;
}
```

Installing a Handler Example 1 (cont.)

[Demo of testsignal.c]

Installing a Handler Example 2

Program testsignalall.c:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <assert.h>
#include <signal.h>

static void myHandler(int iSig) {
    printf("In myHandler with argument %d\n", iSig);
}
...
```

Installing a Handler Example 2 (cont.)

Program testsignalall.c (cont.):

```
...
int main(void) {
    void (*pfRet)(int);
    pfRet = signal(SIGHUP, myHandler); /* 1 */
    pfRet = signal(SIGINT, myHandler); /* 2 */
    pfRet = signal(SIGQUIT, myHandler); /* 3 */
    pfRet = signal(SIGILL, myHandler); /* 4 */
    pfRet = signal(SIGTRAP, myHandler); /* 5 */
    pfRet = signal(SIGABRT, myHandler); /* 6 */
    pfRet = signal(SIGBUS, myHandler); /* 7 */
    pfRet = signal(SIGFPE, myHandler); /* 8 */
    pfRet = signal(SIGKILL, myHandler); /* 9 */
    ...
}
```

Installing a Handler Example 2 (cont.)

Program testsignalall.c (cont.):

```
...
/* Etc., for every signal. */

printf("Entering an infinite loop\n");
for (;;)
    ;
return 0;
}
```

Installing a Handler Example 2 (cont.)

[Demo of testsignalall.c]

Installing a Handler Example 3

Program generates lots of temporary data

- Stores the data in a temporary file
- Must delete the file before exiting

```
...  
int main(void) {  
    FILE *psFile;  
    psFile = fopen("temp.txt", "w");  
    ...  
    fclose(psFile);  
    remove("temp.txt");  
    return 0;  
}
```

Example 3 Problem

What if user types Ctrl-c?

- OS sends a 2/SIGINT signal to the process
- Default handler of 2/SIGINT exits the process

Problem: The temporary file is not deleted

- Process dies before `remove("tmp.txt")` is executed

Challenge: Ctrl-c could happen at any time

- Which line of code will be interrupted???

Solution: Install a signal handler

- Define a “clean up” function to delete the file
- Install the function as a signal handler for 2/SIGINT

Example 3 Solution

```
...
static FILE *psFile; /* Must be global. */
static void cleanup(int iSig) {
    fclose(psFile);
    remove("tmp.txt");
    exit(EXIT_FAILURE);
}
int main(void) {
    void (*pfRet)(int);
    psFile = fopen("temp.txt", "w");
    pfRet = signal(SIGINT, cleanup);
    ...
    raise(SIGINT);
    return 0; /* Never get here. */
}
```

Handling Signals

- The kernel handles signals in the context of the process that receives them, so **a process must run to handle signals.**
- There are three types of handling for signals:
 - The process **exits**. (default for some signals)
 - The process **ignores**. (default for some other signals)
 - The process executes a **signal handler**:

```
oldfun = signal(signum, newfun) ;
```

Signal Handlers – Example

(Note: “signal” is deprecated!)

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void catch_int(int sig_num) {
    signal(SIGINT, catch_int); //install again!
    printf("Don't do that\n");
    fflush(stdout);
}

int main(int argc, char* argv[]) {
    signal(SIGINT, catch_int);
    for ( ;; )
        pause(); //wait until receives a signal.
}
```

Handling Signals (cont.)

- There are some signals that the process cannot catch.
 - For example: `KILL` and `STOP`.
- If you install no signal handlers of your own, the runtime environment sets up a set of **default signal handlers**.
 - For example:
 - The default signal handler for `TERM` calls `exit()`.
 - The default handler for `ABRT` is to dump the process's memory image into a file, and then exit.

Pre-defined Signal Handlers

- There are two pre-defined signal handler functions that we can use:, instead of writing our own.
 - `SIG_IGN`: Causes the process to ignore the specified signal.
 - `signal(SIGINT, SIG_IGN);`
 - `SIG_DFL`: Causes the system to set the default signal handler for the given signal.
 - `signal(SIGTSTP, SIG_DFL);`

Predefined Signal Handler: SIG_IGN

Pre-defined signal handler: SIG_IGN

Can install to ignore signals

```
int main(void) {  
    void (*pfRet)(int);  
    pfRet = signal(SIGINT, SIG_IGN);  
    ...  
}
```

Subsequently, process will ignore 2/SIGINT signals

Predefined Signal Handler: SIG_DFL

Pre-defined signal handler: SIG_DFL

Can install to restore default signal handler

```
int main(void) {  
    void (*pfRet)(int);  
    pfRet = signal(SIGINT, somehandler);  
    ...  
    pfRet = signal(SIGINT, SIG_DFL);  
    ...  
}
```

Subsequently, process will handle 2/SIGINT signals using the default handler for 2/SIGINT signals

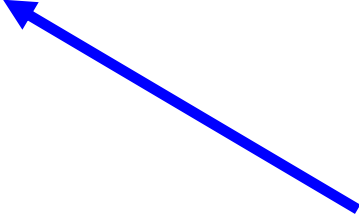
Race Conditions in Signal Handlers

A **race condition** is a flaw in a program whereby the correctness of the program is critically dependent on the sequence or timing of other events.

Race conditions can occur in signal handlers...

Race Condition Example

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    iTemp = iSavingsBalance;  
    iTemp += iMonthlySalary;  
    iSavingsBalance = iTemp;  
}
```



Handler for hypothetical
“update monthly salary” signal

Race Condition Example (cont.)

(1) Signal arrives; handler begins executing

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    ↓ iTemp = iSavingsBalance; 2000  
    iTemp += iMonthlySalary;  
    iSavingsBalance = iTemp;  
}
```

Race Condition Example (cont.)

(2) Another signal arrives; first instance of handler is interrupted; second instance of handler begins executing

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    ↓ iTemp = iSavingsBalance; 2000  
    iTemp += iMonthlySalary;  
    iSavingsBalance = iTemp;  
}
```

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    ↓ iTemp = iSavingsBalance; 2000  
    iTemp += iMonthlySalary;  
    iSavingsBalance = iTemp;  
}
```

Race Condition Example (cont.)

(3) Second instance executes to completion

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    ↓ iTemp = iSavingsBalance; 2000  
    iTemp += iMonthlySalary;  
    iSavingsBalance = iTemp;  
}
```

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    iTemp = iSavingsBalance; 2000  
    iTemp += iMonthlySalary; 2050  
    ↓ iSavingsBalance = iTemp; 2050  
}
```

Race Condition Example (cont.)

(4) Control returns to first instance, which executes to completion

```
void addSalaryToSavings(int iSig) {  
    int iTemp;  
    iTemp = iSavingsBalance; 2000  
    iTemp += iMonthlySalary; 2050  
    iSavingsBalance = iTemp; 2050  
}
```



Lost 50 !!!

Blocking Signals in Handlers

Blocking signals

- To **block** a signal is to **queue** it for delivery at a later time

Why block signals when handler is executing?

- Avoid race conditions when another signal of type x occurs while the handler for type x is executing

How to block signals when handler is executing?

- **Automatic** during execution of signal handler!!!
- Previous sequence **cannot happen!!!**
- While executing a handler for a signal of type x, all signals of type x are blocked
- When/if signal handler returns, block is removed

Race Conditions in General

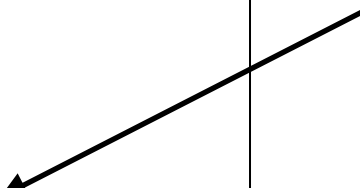
Race conditions can occur elsewhere too

```
int iFlag = 0;

void myHandler(int iSig) {
    iFlag = 1;
}

int main(void) {
    if (iFlag == 0) {
        /* Do something */
    }
}
```

Problem: `myflag` might become 1 just after the comparison!



Must make sure that **critical sections** of code are not interrupted

Blocking Signals in General

How to block signals in general?

- Not possible in C90
- Possible using POSIX functions...

POSIX Signal Handling

C90 standard

- Defines **signal()** and **raise()** functions
 - Work across all systems (UNIX, LINUX, Windows), but...
 - Work **differently** across some systems!!!
 - On some systems, signals are blocked during execution of handler for that type of signal -- but not so on other (older) systems
 - On some (older) systems, handler installation for signals of type x is cancelled after first signal of type x is received; must reinstall the handler -- but not so on other systems
- Does not provide mechanism to block signals in general

POSIX Signal Handling

POSIX standard

- Defines **kill()**, **sigprocmask()**, and **sigaction()** functions
 - Work the same across all POSIX-compliant UNIX systems (Linux, Solaris, etc.), but...
 - Do not work on non-UNIX systems (e.g. Windows)
- Provides mechanism to block signals in general

Blocking Signals in General

Each process has a signal mask in the kernel

- OS uses the mask to decide which signals to deliver
- User program can modify mask with **sigprocmask()**

sigprocmask()

```
int sigprocmask(int iHow,  
                const sigset_t *psSet,  
                sigset_t *psOldSet);
```

- **psSet**: Pointer to a signal set
- **psOldSet**: (Irrelevant for our purposes)
- **iHow**: How to modify the signal mask
 - **SIG_BLOCK**: Add **psSet** to the current mask
 - **SIG_UNBLOCK**: Remove **psSet** from the current mask
 - **SIG_SETMASK**: Install **psSet** as the signal mask
- Returns 0 iff successful

Functions for constructing signal sets

- **sigemptyset()**, **sigaddset()**, ...

Note: No parallel function in C90

Blocking Signals Example

```
sigset_t sSet;
int main(void) {
    int iRet;
    sigemptyset(&sSet);
    sigaddset(&sSet, SIGINT);
    iRet = sigprocmask(SIG_BLOCK, &sSet, NULL);
    assert(iRet == 0);
    if (iFlag == 0) {
        /* Do something */
    }
    iRet = sigprocmask(SIG_UNBLOCK, &sSet, NULL);
    assert(iRet == 0);
    ...
}
```

Blocking Signals in Handlers

Signals of type `x` automatically are blocked when executing handler for signals of type `x`

Additional signal types to be blocked can be defined at time of handler installation...

Installing a Signal Handler

`sigaction()`

```
int sigaction(int iSig,  
              const struct sigaction *psAction,  
              struct sigaction *psOldAction);
```

- **iSig**: The type of signal to be affected
- **psAction**: Pointer to a structure containing instructions on how to handle signals of type **iSig**, including signal handler name and which signal types should be blocked
- **psOldAction**: (Irrelevant for our purposes)
- Installs an appropriate handler
- Automatically blocks signals of type **iSig**
- Returns 0 iff successful

Note: More powerful than C90 `signal()`

Installing a Handler Example

Program testsigaction.c:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

static void myHandler(int iSig) {
    printf("In myHandler with argument %d\n", iSig);
}

...
```

Installing a Handler Example (cont.)

Program testsigaction.c (cont.):

```
...
int main(void) {
    int iRet;
    struct sigaction sAction;
    sAction.sa_flags = 0;
    sAction.sa_handler = myHandler;
    sigemptyset(&sAction.sa_mask);
    iRet = sigaction(SIGINT, &sAction, NULL);
    assert(iRet == 0);

    printf("Entering an infinite loop\n");
    for (;;)
        ;
    return 0;
}
```


Installing a Handler Example (cont.)

[Demo of testsigaction.c]

Predefined Signals

List of the predefined signals:

```
$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL
5) SIGTRAP     6) SIGABRT     7) SIGBUS      8) SIGFPE
9) SIGKILL     10) SIGUSR1    11) SIGSEGV    12) SIGUSR2
13) SIGPIPE    14) SIGALRM    15) SIGTERM    17) SIGCHLD
18) SIGCONT    19) SIGSTOP    20) SIGTSTP    21) SIGTTIN
22) SIGTTOU    23) SIGURG     24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH    29) SIGIO
30) SIGPWR     31) SIGSYS     34) SIGRTMIN    35) SIGRTMIN+1
36) SIGRTMIN+2 37) SIGRTMIN+3 38) SIGRTMIN+4 39) SIGRTMIN+5
40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8 43) SIGRTMIN+9
44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13
52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9
56) SIGRTMAX-8  57) SIGRTMAX-7  58) SIGRTMAX-6  59) SIGRTMAX-5
60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2  63) SIGRTMAX-1
64) SIGRTMAX
```

Applications can define their own signals

- An application can define signals with unused values

Intermediate Summary

- Each signal may have a **signal handler**, which is a function that gets called when the process receives that signal.
- If a signal is sent to the process, the next time the process runs, the operating system causes the process to **run the signal handler**, no matter what it was doing before.
- When that signal handler function returns, the process **continues execution** from wherever it happened to be before the signal was received.

Avoiding Signal Races – Masking Signals

- Because signals are handled **asynchronously**, race conditions can occur:
 - A signal may be received and handled in the middle of an operation that should not be interrupted;
 - A second signal may occur before the current signal handler finished.
 - The second signal may be of a different type or of the same type as the first one.
- Therefore we need to **block signals** from being processed when they are harmful.
 - The blocked signal will be processed after the block is removed.
 - Some signals cannot be blocked.

Sigprocmask

Allows to specify a set of signals to block, and/or get the list of signals that were previously blocked.

```
int sigprocmask(int how, const sigset_t *set,  
                sigset_t *oldset)
```

1. `int how`:

- **Add** (SIG_BLOCK)
- **Delete** (SIG_UNBLOCK)
- **Set** (SIG_SETMASK).

2. `const sigset_t *set`:

- The set of signals.

3. `sigset_t *oldse`:

- If not NULL, the previous mask will be returned.

```
sigset_t set;  
  
sigemptyset(&set);  
sigaddset(&set, SIGINT);  
sigaddset(&set, SIGTERM);  
sigprocmask(SIG_SETMASK, &set, NULL);  
//blocked signals: SIGINT and SIGTERM  
  
sigemptyset(&set);  
sigaddset(&set, SIGINT);  
sigaddset(&set, SIGALRM);  
sigprocmask(SIG_BLOCK, &set, NULL);  
//blocked signals: SIGINT, SIGTERM, SIGALRM  
  
sigemptyset(&set);  
sigaddset(&set, SIGTERM);  
sigaddset(&set, SIGUSR1);  
sigprocmask(SIG_UNBLOCK, &set, NULL);  
//blocked signals: SIGINT and SIGALRM
```

Handling Signals

- So far we saw two system calls:
 - `signal`
 - Installs a signal for a single use,
 - Must reinstall each time we get a signal!
 - **Deprecated!**
 - `sigprocmask`
 - Defines which signals to block,
 - Must be called in each signal handler to block and release signals.

Sigaction

```
int sigaction(int sig,  
              struct sigaction *new_act,  
              struct sigaction *old_act);
```

- Allows the calling process to examine and/or specify the **action** to be associated with a specific signal.
 - **action** = signal handler+signal mask+flags

Sigaction cont.

- The signal mask is calculated and installed **only for the duration of the signal handler.**
- By default, the signal “sig” is also blocked when the signal occurs.
- Once an action is installed for a specific signal using `sigaction`, **it remains installed** until another action is explicitly requested.


```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void termination_handler(int signum) {
    exit(7);
}

int main (void) {
    struct sigaction new_action, old_action;
    new_action.sa_handler =
        termination_handler;
    sigemptyset(&new_action.sa_mask);
    sigaddset(&new_action.sa_mask, SIGTERM);
    new_action.sa_flags = 0;
    sigaction(SIGINT, NULL, &old_action);
    if (old_action.sa_handler != SIG_IGN) {
        sigaction(SIGINT, &new_action, NULL);
    }
    sleep(10);
    return 0;
}
```

Signals: Summary

- Signals are **notifications** sent to a process
- The OS causes the process to handle a signal **immediately** the next time it runs.
- There are **default signal handlers** for processes.
- These handlers can be changed using `signal` or `sigaction`.
- To avoid race conditions, one usually needs to block signals some of the time using `sigprocmask` and/or `sigaction`.

Summary

Signals

- A **signal** is an asynchronous event mechanism
- C90 **raise()** or POSIX **kill()** sends a signal
- C90 **signal()** or POSIX **sigaction()** installs a signal handler
 - Most predefined signals are “catchable”
- Beware of race conditions
- Signals of type x automatically are blocked while handler for type x signals is running
- POSIX **sigprocmask()** blocks signals in any critical section of code

Alarms

`alarm()`

`unsigned int alarm(unsigned int uiSec) ;`

- Sends 14/SIGALRM signal after `uiSec` seconds
- Cancels pending alarm if `uiSec` is 0
- Uses **real time**, alias **wall-clock time**
 - Time spent executing other processes counts
 - Time spent waiting for user input counts
- Return value is meaningless

Used to implement time-outs



Alarm Example 1

Program testalarm.c:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <assert.h>
#include <signal.h>
#include <unistd.h>

static void myHandler(int iSig) {
    printf("In myHandler with argument %d\n", iSig);

    /* Set another alarm. */
    alarm(2);
}

...
```

Alarm Example 1 (cont.)

Program testalarm.c (cont.):

```
int main(void)
{
    void (*pfRet) (int);
    sigset_t sSet;
    int iRet;

    /* Make sure that SIGALRM is not blocked. */
    sigemptyset(&sSet);
    sigaddset(&sSet, SIGALRM);
    iRet = sigprocmask(SIG_UNBLOCK, &sSet, NULL);
    assert(iRet == 0);

    pfRet = signal(SIGALRM, myHandler);
    assert(pfRet != SIG_ERR);
    ...
}
```

Alarm Example 1 (cont.)

Program testalarm.c (cont.):

```
...  
  
    /* Set an alarm. */  
    alarm(2);  
  
    printf("Entering an infinite loop\n");  
    for (;;)   
        ;  
  
    return 0;  
}
```

Alarm Example 1 (cont.)

[Demo of testalarm.c]

Alarm Example 2

Program `testalarm_timeout.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <signal.h>
#include <unistd.h>

static void myHandler(int iSig)
{
    printf("\nSorry.  You took too long.\n");
    exit(EXIT_FAILURE);
}
```

Alarm Example 2 (cont.)

Program testalarmtimeout.c (cont.):

```
int main(void) {
    int i;
    void (*pfRet)(int);
    sigset_t sSet;
    int iRet;

    /* Make sure that SIGALRM is not blocked. */
    sigemptyset(&sSet);
    sigaddset(&sSet, SIGALRM);
    iRet = sigprocmask(SIG_UNBLOCK, &sSet, NULL);
    assert(iRet == 0);
    ...
}
```

Alarm Example 2 (cont.)

Program testalarmtimeout.c (cont.):

```
    pfRet = signal(SIGALRM, myHandler);  
    assert(pfRet != SIG_ERR);  
  
    printf("Enter a number:  ");  
    alarm(5);  
    scanf("%d", &i);  
    alarm(0);  
  
    printf("You entered the number %d.\n", i);  
    return 0;  
}
```

Alarm Example 2 (cont.)

[Demo of testalarmtimeout.c]

Interval Timers

`settimer()`

```
int settimer(int iWhich,  
             const struct itimerval *psValue,  
             struct itimerval *psOldValue);
```

- Sends 27/SIGPROF signal continually
- Timing is specified by **psValue**
- **psOldValue** is irrelevant for our purposes
- Uses **virtual time**, alias **CPU time**
 - Time spent executing other processes does not count
 - Time spent waiting for user input does not count
- Returns 0 iff successful

Used by execution profilers

Interval Timer Example

Program testtimer.c:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <signal.h>
#include <sys/time.h>

static void myHandler(int iSig) {
    printf("In myHandler with argument %d\n", iSig);
}
...
```

Interval Timer Example (cont.)

Program testtimer.c (cont.):

```
int main(void)
{
    int iRet;
    void (*pfRet) (int);
    struct itimerval sTimer;

    pfRet = signal(SIGPROF, myHandler);
    assert(pfRet != SIG_ERR);
    ...
}
```

Interval Timer Example (cont.)

Program testtimer.c (cont.):

```
...
/* Send first signal in 1 second, 0 microseconds. */
sTimer.it_value.tv_sec = 1;
sTimer.it_value.tv_usec = 0;

/* Send subsequent signals in 1 second,
   0 microseconds intervals. */
sTimer.it_interval.tv_sec = 1;
sTimer.it_interval.tv_usec = 0;

iRet = setitimer(ITIMER_PROF, &sTimer, NULL);
assert(iRet != -1);

printf("Entering an infinite loop\n");
for (;;)
    ;
return 0;
}
```


Interval Timer Example (cont.)

[Demo of testitimer.c]

Summary

Alarms

- Call **alarm()** to deliver 14/SIGALRM signals in real/wall-clock time
- Alarms can be used to implement time-outs

Interval Timers

- Call **setitimer()** to deliver 27/SIGPROF signals in virtual/CPU time
- Interval timers are used by execution profilers